

MARY SHARNICE OTIENO C027-01-2727/2024

Question 1: Design Decisions

(i) My admission number is C027-01-2727/2024. The xxxx portion (2727) has a last digit of 7, so I created $5 + 7 = 12$ gigs. Since 2727 is odd, my categories are Marketing, Data, and Consulting, and since the first two digits of 2727 (27) are 10 or more, my currency is USD.

(ii) I included the status field because gigs go through a real lifecycle on a freelancing platform — a client posts a job, a freelancer starts working on it, and eventually it wraps up. The three possible statuses are "Open" (available for freelancers to apply), "In Progress" (a freelancer is actively working on it), and "Closed" (the gig is finished or no longer available).

Question 2: Validation

(i) The budget field uses Pydantic's `Field(..., gt=0)`, which requires the value to be a float strictly greater than zero. This matters because a gig with a zero or negative budget wouldn't make sense in the real world, so this rule stops bad data from ever entering the system.

(ii) I made category a Python Enum (`CategoryEnum`) restricted to my three assigned values (Marketing, Data, Consulting) instead of a plain string. Pydantic automatically validates any incoming category against this Enum and rejects the request with an error if the value isn't one of the three allowed options.

Question 3: Error Handling

(i) My API returns a 404 Not Found status code when someone requests a gig ID that doesn't exist in `gigs_db`. This is the correct code because the client's request itself is well-formed — the problem is that the specific resource they asked for simply isn't there.

(ii) When a user submits invalid data to create a gig (e.g., a title that's too short or a negative budget), FastAPI and Pydantic automatically return a 422 Unprocessable Entity status code. This is appropriate because the request reached the server fine, but the data inside it failed validation against the rules defined in the `GigCreate` model.

Question 4: Search and Filtering

(i) The `/gigs/search` endpoint takes a query parameter `q` and checks whether that string appears (case-insensitively) inside each gig's title, returning only the matching gigs. If no titles match, it simply returns an empty list rather than an error, since "no results" is a valid outcome of a search, not a failure.

(ii) The `/gigs` endpoint accepts three optional query parameters: `category`, `min_budget`, and `max_budget`. If provided, each filter progressively narrows down the full gig list — `category` checks for an exact match against the assigned categories, while `min_budget` and `max_budget` filter gigs whose budget falls within that range; if none are provided, it just returns every gig.

Question 5: Code Walkthrough

(i) The PUT /gigs/{gig_id} endpoint updates an existing gig's budget and/or status using the GigUpdate model, where both fields are optional so a client can update just one without touching the other. If the gig_id doesn't match any gig in gigs_db, the endpoint raises an HTTPException with a 404 status code instead of trying to update something that doesn't exist.

(ii) The POST /gigs endpoint creates a new gig using the GigCreate model, which validates the title, description, category, budget, and client_name before anything is added to the database. It then automatically assigns the next available ID, sets the currency to the platform's fixed USD value, defaults the status to "Open", and appends the new gig to gigs_db, returning the completed gig object with a 201 Created status code.